

High-Fidelity Schema Research Project

A Plain-English Walkthrough of the Complete Architecture

Audience: Readers with no specialist background

Reading level: Middle-school friendly

Project snapshot: July 29, 2026

The project tries to produce the strongest schema supported by real evidence. When evidence is weak, the system should say “unknown,” “conflicted,” “partial,” or “unsupported” instead of making a confident guess.

This document explains the deterministic product, the semantic A/B/C/D study, the NDP-50 external-validation track, and the human governance system as one connected research architecture.

Contents

The guide moves from the full project map to the extraction product, semantic research, NDP-50, governance, repository structure, and current status.

How to Use This Guide	5
1. The Whole Project in One Sentence	5
2. The Big Project Map	6
3. What “High-Fidelity” Means	6
4. What “Deterministic-First” Means	7
5. Final Deliverable One: The Dataset-to-Schema Product	7
6. Intake: Checking the Input	8
7. Format Detection: Deciding What Kind of File It Is	8
8. The Capability Registry	9
9. Extractors: Format-Specific Readers	9
9.1 CSV and TSV	9
9.2 HDF5	10
9.3 NetCDF/CF	10
9.4 Zarr	10
9.5 Parquet and Arrow	10
9.6 JSON and JSON Lines	11
9.7 XML and XSD	11
9.8 Opaque Binary Data	11
10. Why the Schema Has Layers	11
10.1 Physical Layer	11
10.2 Logical Layer	12
10.3 Semantic Layer	12
10.4 Unit Layer	12
10.5 Temporal Layer	12
10.6 Evidence Layer	13
10.7 Provenance Layer	13
11. Structured Outcomes and Honest Failure	13
12. The Unified Schema Envelope	14
13. Deterministic Validators	14

14. Final Deliverable Two: The Semantic Architecture Study	15
15. The Four A/B/C/D Architectures	15
15.1 Architecture A: Deterministic Only	15
15.2 Architecture B: One Dataset-Level Response	16
15.3 Architecture C: B Plus Deterministic Verification	16
15.4 Architecture D: Historical Complex Scheduling	16
16. What the A/B/C/D Study Measures	16
16.1 Verified Claim Coverage	16
16.2 Selective Risk	16
16.3 Unsupported Claim Rate	17
16.4 Resource Cost	17
17. Independent Gold: The Reference Standard	17
18. Blindness and Information Separation	17
19. Annotator Calibration	18
20. Semantic Backends	18
21. Backend Qualification	19
22. Power Analysis and the Statistical Unit	19
23. The NDP-50 External-Validation Track	20
24. Building the NDP Candidate Frame	20
25. Selecting and Splitting the NDP-50	20
26. What Each NDP Split May Be Used For	21
26.1 Development: 15 Datasets	21
26.2 Validation: 10 Datasets	21
26.3 Sealed Test: 25 Datasets	21
27. Capability Debt	22
28. Declared Versus Observed	22
29. The CPA Sub-Study	23
30. Why the Project Has Human Gates	23
31. The Major Human and External Gates	24
32. Why Gate Order Matters	24
33. Freeze, Hashes, and Content Addressing	24

34. Preregistration	25
35. Test-Release Authorization	26
36. The Frozen Artifact Paper and Project History	26
37. Post-Freeze Engineering: Phases 12-20	26
38. Agent-Ready, Not an Autonomous Data Agent	27
39. Why the Product and the Study Stay Separate	27
40. Repository Map	28
41. The Project's Current State	28
41.1 Built or Structurally Ready	28
41.2 Still Incomplete	29
42. Where Swathi's Review Fits	29
43. A School-Exam Analogy	30
44. Compact Glossary	31
45. Final Summary	31
Source Documents in This Repository	32

How to Use This Guide

This project contains a software product, several research experiments, an external NDP-50 validation track, and a set of human review gates. Those pieces share the same foundation, but they do not mean the same thing.

This guide explains:

- what each major component is;
- why the component exists;
- how information moves through the project;
- where humans must make decisions;
- what has already been built;
- what is still incomplete; and
- what the project is allowed, and not allowed, to claim.

The most important idea is simple:

The project tries to produce the strongest schema supported by real evidence. When the evidence is weak, the system should say “unknown,” “conflicted,” “partial,” or “unsupported” instead of making a confident guess.

1. The Whole Project in One Sentence

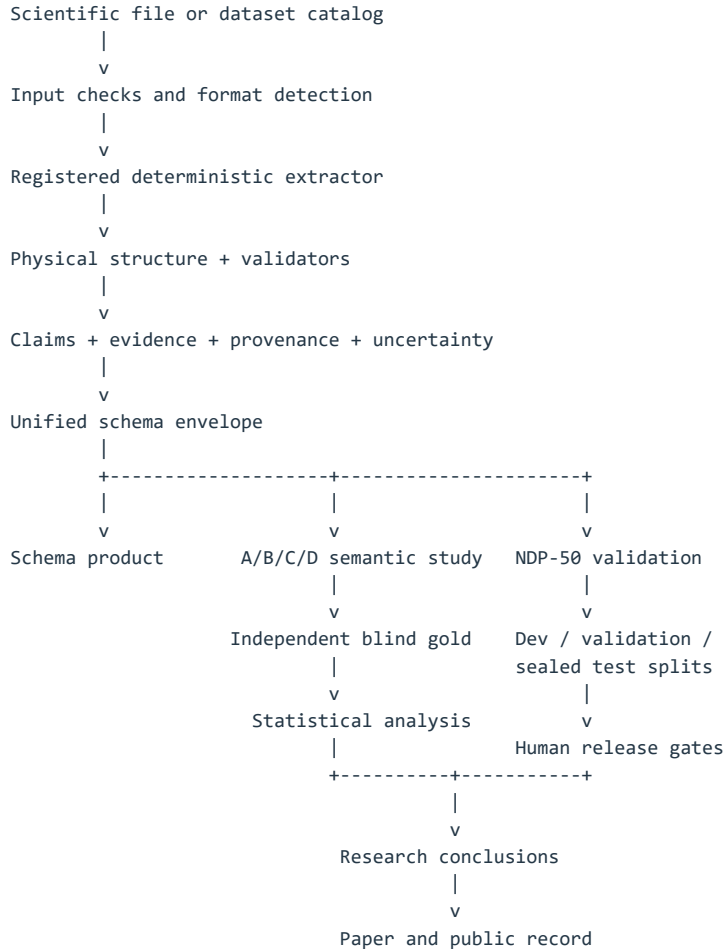
The project takes scientific data files, extracts their structure with deterministic software, records the evidence for every important claim, and then carefully studies whether limited semantic-model reasoning adds reliable value without creating unsupported information.

It is not simply “an AI that guesses schemas.”

It is better understood as a research factory with four connected parts:

1. a deterministic dataset-to-schema product;
 2. a controlled semantic-architecture study;
 3. the NDP-50 external-validation track; and
 4. human review, governance, preregistration, and publication controls.
-

2. The Big Project Map



Every arrow is a boundary. A later stage may use information from an earlier stage, but it may not silently rewrite what the earlier stage actually observed.

3. What “High-Fidelity” Means

“High-fidelity” means faithful to the source.

Imagine a file contains:

```
temp = 35
```

A guessing system might say:

```
temp is air temperature measured in degrees Celsius
```

But the file did not tell us:

- whether `temp` means temperature;
- whether it is air, water, soil, or machine temperature;
- whether 35 uses Celsius, Fahrenheit, or another scale; or
- whether `temp` is short for “temporary.”

A high-fidelity system should instead say something like:

```
Observed field: temp
Observed type: numeric
Semantic meaning: unknown
```

Unit: unknown

Reason: not enough supporting evidence

The second answer is less exciting, but it is more trustworthy.

The project therefore treats these ideas as different:

Idea	Plain meaning
More output	The system says more things
Better output	The system says more correct and supported things
High confidence	The system sounds sure
Strong evidence	The source actually supports the claim

Confidence is not the same as evidence.

4. What “Deterministic-First” Means

“Deterministic” means that the same input, program version, and settings should produce the same result.

For example, if the first row of a CSV file is:

```
student_id,age,height_cm
```

the program can directly observe three physical fields:

- `student_id`;
- `age`; and
- `height_cm`.

No language model is needed to discover that those columns physically exist.

“Deterministic-first” means the project follows this order:

1. read the real file structure with a format-aware parser;
2. apply explicit rules for time, units, profiles, and conflicts;
3. record evidence and uncertainty;
4. only then, in the research study, allow tightly limited semantic reasoning; and
5. never allow semantic reasoning to invent a physical field.

The purpose is to place file facts before model guesses.

5. Final Deliverable One: The Dataset-to-Schema Product

The first final deliverable is a practical software product.

Its input is a supported scientific data resource. Its output is a structured, evidence-grounded schema.

The product does not need a semantic model to work. Even if the research study later finds that models add no reliable value, this product can still be useful.

The canonical flow is:

```
File or local directory store
-> ExtractionRequest
-> input validation
-> format signals
-> format decision
-> capability registry
```

```
-> registered extractor  
-> deterministic validators  
-> ExtractionOutcome  
-> unified schema envelope
```

The main program entry points are:

```
extract_path(ExtractionRequest(...))  
python -m high_fidelity_schema_study.cli extract <path>
```

These names are implementation details. The important idea is that there is one controlled front door for extraction.

6. Intake: Checking the Input

“Intake” is the front desk of the system.

Before trying to understand a file, it checks:

- whether the path exists;
- whether the input is a file or a directory store;
- whether the input is readable;
- whether the requested limits are valid;
- whether a format hint was supplied; and
- whether the resource is within the allowed safety boundaries.

The purpose is to stop invalid or unsafe work before a parser begins.

For remote NDP resources, the surrounding acquisition workflow also records:

- the original URL;
- redirects and the final URL;
- content type;
- transferred byte count;
- file hash;
- attempt count;
- failure reason; and
- resource and decompression limits.

Unknown size never means unlimited download.

7. Format Detection: Deciding What Kind of File It Is

The system cannot trust a filename extension by itself.

A file named `experiment.csv` might actually contain a ZIP archive or an HDF5 container. The project therefore combines several format signals:

- magic bytes or container signatures;
- directory markers such as Zarr metadata files;
- a compatible user-provided format hint;
- filename suffixes;
- MIME information; and
- conservative text probes.

The rough priority is:


```
strong internal signature
  > compatible explicit hint
  > suffix or bounded text probe
```

If strong signals disagree, the system should produce a structured conflict or abstention. It should not pick the most convenient answer.

The purpose is to prevent the wrong parser from producing a convincing but false schema.

8. The Capability Registry

The capability registry is a directory of supported extraction abilities.

It records:

- which formats are supported;
- which extractor handles each format;
- what dependencies are required;
- what the extractor can observe;
- what the extractor does not claim to support; and
- which version or implementation produced the result.

Think of it as a list of trained translators:

Data format	Specialist
CSV/TSV	Table-structure extractor
HDF5	Hierarchical-container extractor
NetCDF/CF	Scientific-array and coordinate extractor
Zarr v2	Local directory-store metadata extractor
Parquet/Arrow	Columnar-schema and footer extractor
JSON/JSON Lines	Bounded nested-structure extractor
XML/XSD	Namespace-aware structure extractor

If the registry has no specification-backed extractor for a resource, the correct outcome is an abstention or unsupported result, not a guessed schema.

9. Extractors: Format-Specific Readers

An extractor is the program that reads one kind of data format.

Different formats need different extractors because they organize information differently.

9.1 CSV and TSV

CSV is table-shaped. The extractor can inspect:

- headers;
- a bounded number of sample rows;
- conservative physical types;
- missing values;
- value ranges;
- unique-value ratios;

- possible identifiers;
- possible time fields; and
- guarded unit or semantic hints.

It does not read an unlimited number of rows just because the file is large.

9.2 HDF5

HDF5 behaves more like folders inside folders. The extractor preserves:

- groups;
- datasets;
- full paths;
- dtypes;
- shapes; and
- explicit attributes.

9.3 NetCDF/CF

NetCDF/CF is common in scientific array data. The extractor can preserve:

- dimensions;
- variables;
- coordinate relationships;
- calendars;
- units;
- standard names; and
- fill or missing-value markers.

9.4 Zarr

The current registered support is intentionally limited to local Zarr v2 directory-store metadata.

It does not claim:

- arbitrary remote Zarr support;
- Zarr v3 support;
- full Xarray reconstruction; or
- inspection of every data chunk.

9.5 Parquet and Arrow

The extractor reads schema and footer information such as:

- nested field paths;
- nullability;
- row groups;
- encodings;
- compression;
- embedded metadata; and
- footer statistics.

It does not need to read all row values.

9.6 JSON and JSON Lines

JSON may contain nested objects and arrays. The extractor reports bounded observed paths and keeps two ideas separate:

- a declared JSON Schema; and
- the structure observed in examples.

Observed examples do not prove that every possible record has the same structure.

9.7 XML and XSD

The extractor reads lightweight namespace-aware XML structure and selected XSD information.

It is not presented as a complete validating XML Schema processor.

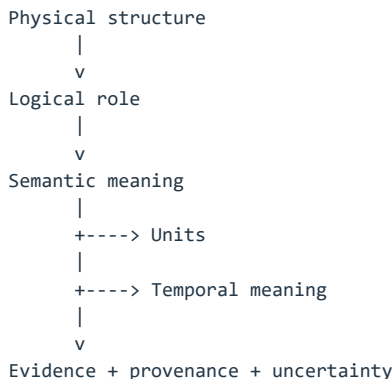
9.8 Opaque Binary Data

If the system cannot identify a specification-backed structure, it deliberately abstains.

The purpose is to avoid converting random bytes into fictional fields.

10. Why the Schema Has Layers

A schema is not one flat answer. The project separates different kinds of claims so that a strong physical observation is not confused with a weak semantic guess.



10.1 Physical Layer

The physical layer records what is directly present:

- field names;
- field paths;
- groups;
- variables;
- dimensions;
- dtypes;
- shapes;
- nullability;
- missing markers;
- attributes; and
- format-specific structure.

It answers:

What physically exists in this resource?

10.2 Logical Layer

The logical layer describes a field's job in the data organization.

Examples include:

- identifier;
- coordinate;
- timestamp;
- measurement;
- category; and
- relationship key.

It answers:

What role does this field appear to play?

10.3 Semantic Layer

The semantic layer describes the real-world concept.

Examples include:

- air temperature;
- latitude;
- longitude;
- precipitation;
- patient identifier; and
- species name.

This layer is harder because short names can be ambiguous. A field called `ch1` might mean chlorophyll, cholesterol, channel, or something specific to one instrument.

It answers:

What does this field mean in the world?

10.4 Unit Layer

The unit layer records and normalizes supported units.

Examples include:

- metre;
- second;
- degree Celsius;
- kilogram; and
- millimetres per day.

A suffix is not enough evidence by itself. For example, the `c` in `input_dim_c` may mean “channel,” not Celsius.

10.5 Temporal Layer

The temporal subsystem handles more than “this column looks like a date.”

It can preserve:

- UTC;
- explicit time-zone offsets;
- time values with no zone;
- mixed offsets;
- frequency;
- regularity;
- missing intervals;
- split year/month/day fields;
- competing time-axis candidates; and
- special scientific calendars.

10.6 Evidence Layer

Evidence is the specific support for a claim.

For example:

```
Claim: lat represents latitude
Evidence:
- field name is lat
- standard_name attribute is latitude
- units attribute is degrees_north
```

It answers:

Why should anyone believe this claim?

10.7 Provenance Layer

Provenance records the history of a result:

- source file identity;
- source hash;
- extractor identity;
- program version;
- rules or validators used;
- processing time; and
- the step that created the claim.

Evidence explains why a claim is supported. Provenance explains where the claim came from and how it was produced.

11. Structured Outcomes and Honest Failure

The system has more than two outcomes.

Outcome	Plain meaning
success	The requested extraction completed
partial	Some useful structure was extracted, but part of the task could not be completed
failed	The system tried and failed
abstained	The system recognized that it should not guess
unknown	A specific property cannot be determined
conflicted	Two or more pieces of evidence disagree

Outcome	Plain meaning
unsupported	The current registered product does not support the feature

This is important because a hidden failure is dangerous. A structured failure can be counted, studied, and repaired.

12. The Unified Schema Envelope

Each file format has its own natural structure. CSV has columns, HDF5 has groups and datasets, NetCDF has variables and dimensions, XML has a tag tree, and JSON has nested paths.

The unified schema envelope places those different results inside a shared outer structure:

- resource identity;
- format signals and decision;
- extractor capability;
- physical structure;
- logical roles;
- semantic hints;
- units;
- temporal semantics;
- claims;
- evidence;
- provenance;
- issues;
- conflicts;
- abstentions; and
- unsupported features.

The envelope is an additive projection. It does not create new semantic truth.

Its purpose is to let downstream tools inspect different formats through one consistent interface.

13. Deterministic Validators

A validator is a rule-based checker.

The project includes validators for:

- schema shape;
- time semantics;
- unit normalization;
- evidence links;
- content hashes;
- experiment configurations;
- human-return payloads;
- release packets;
- execution chronology; and
- statistical replay.

A validator can detect:

- missing files;
- invalid fields;
- incorrect hashes;
- impossible timestamps;
- modified frozen values;
- missing required confirmations;
- inconsistent calculations; and
- broken B/C replay identity.

A validator cannot decide whether a human scientific judgment is wise. It checks whether the judgment was recorded according to the contract.

14. Final Deliverable Two: The Semantic Architecture Study

The deterministic product can observe physical structure, but some scientific meaning remains difficult.

The research study asks:

On independently annotated blind scientific datasets, what do one dataset-level semantic response and deterministic verification add beyond deterministic extraction?

The study does not send an unconstrained raw file to a model and ask it to “make a schema.”

Its input is a frozen deterministic observation-and-task bundle. That boundary protects the physical schema and limits what the semantic model may do.

15. The Four A/B/C/D Architectures

Architecture	What it does	Model-generation cost
A	Deterministic extraction only	No semantic-model call
B	A plus at most one dataset-level semantic response	At most one call per eligible dataset
C	Replays the exact B response and applies deterministic verification	No new generation call
D	Historical per-field or manually grouped scheduling and merge behavior	Usually more complex and more expensive

15.1 Architecture A: Deterministic Only

A uses:

- registered extractors;
- explicit metadata;
- deterministic rules;
- unit and time validators; and
- evidence recording.

It is the control condition.

The purpose is to measure what the product can do without semantic generation.

15.2 Architecture B: One Dataset-Level Response

B starts with A and allows the model to inspect a constrained dataset-level bundle. The model can make at most one semantic response for an eligible dataset.

The purpose is to test whether limited whole-dataset context adds useful meaning more efficiently than many field-by-field calls.

15.3 Architecture C: B Plus Deterministic Verification

C must use the exact response produced for B. It does not ask the model again.

It then checks:

- whether named fields actually exist;
- whether the response is relevant to the requested target;
- whether evidence supports the claim;
- whether explicit metadata conflicts with it;
- whether the unit is plausible;
- whether the label is in the allowed vocabulary; and
- whether the model attempted to create a physical field.

The purpose is to isolate the effect of verification.

If C asked the model again, a different answer could be caused by model randomness rather than verification. Exact replay prevents that confusion.

15.4 Architecture D: Historical Complex Scheduling

D preserves the older per-field or manually grouped semantic scheduling and merge behavior.

It may involve:

- more calls;
- manual grouping;
- a scheduler;
- separate field-level prompts; and
- more complicated merging.

The purpose is to test whether that extra complexity produces enough benefit to justify its cost.

The project does not add a new architecture after seeing results just to rescue a preferred conclusion.

16. What the A/B/C/D Study Measures

16.1 Verified Claim Coverage

This asks:

Of the supported semantic facts that should be found, how many did the system correctly recover?

More coverage is useful only when the claims are correct and supported.

16.2 Selective Risk

This asks:

Among the claims the system chose to make, how often was it wrong?

A conservative system may answer fewer questions but make fewer errors.

16.3 Unsupported Claim Rate

This asks:

How often did the system make a claim that sounded reasonable but lacked acceptable evidence?

This metric is especially important for model-generated semantics.

16.4 Resource Cost

The study also records:

- model calls;
- tokens;
- latency;
- retries;
- failures;
- GPU or hardware requirements; and
- effective cost of verification.

The question is not only “Does it help?” but also “Is the help worth the cost?”

17. Independent Gold: The Reference Standard

“Gold” is the reference answer used to evaluate the architectures.

The final gold cannot simply be copied from NDP metadata or created by the system developers.

The workflow requires:

1. two qualified non-developer annotators;
2. the same pair across the corpus;
3. independent annotation;
4. access to original resources and approved documentation;
5. no access to A/B/C/D outputs;
6. content-hashed independent submissions;
7. disagreement calculation only after both submissions are frozen;
8. human consensus or qualified adjudication; and
9. zero unresolved required slots before approval.

The purpose is to prevent the system’s own predictions from shaping the answer key.

Human consensus is the reference standard for the study. It is not described as a perfect “human upper bound.”

18. Blindness and Information Separation

“Blind” means hiding information that could bias a decision.

Gold annotators must not see:

- architecture outputs;
- which backend produced an answer;

- model-performance summaries; or
- final statistical results.

The model must not receive:

- gold labels;
- gold-only evidence;
- test outcomes; or
- another reviewer's private submission.

Separate reviewers receive separate least-access packets. Independent submissions remain hidden from one another until both hashes are frozen.

The purpose is to prevent answers and evaluation rules from influencing one another.

19. Annotator Calibration

Before annotating the blind or NDP corpus, the final annotator pair must complete a preregistered calibration round.

Calibration is like training two graders on practice work before they grade a final exam.

It checks:

- whether both understand the handbook;
- whether they use the controlled vocabulary correctly;
- whether they record evidence spans;
- whether their agreement meets the frozen threshold; and
- whether the exact final annotator identities match the passing calibration record.

If the handbook or vocabulary changes, previously revealed calibration cases cannot simply be reused to confirm the new version.

The purpose is to show that the annotators can apply the rules reliably before they see formal study cases.

20. Semantic Backends

A backend is the complete system that runs a semantic model.

It includes more than a model family name. A frozen backend record includes:

- checkpoint;
- quantization;
- checkpoint or file hash where available;
- runtime and runtime version;
- model identifier;
- endpoint;
- context limit;
- decoding settings;
- prompt hash;
- response-schema hash;
- hardware and offload configuration; and
- qualification evidence.

The planned local family panel includes GLM, Qwen, and Llama checkpoints. They are sensitivity conditions, not an ensemble and not a voting committee.

One eligible backend must be designated as primary before blind results are inspected.

21. Backend Qualification

Qualification asks whether a backend can reliably participate in the experiment.

It checks:

- endpoint stability;
- valid response format;
- compliance with target scope;
- repeatability under frozen settings;
- context fit;
- truncation;
- timeout and failure rates;
- token and latency feasibility;
- hardware feasibility; and
- exact B/C response replay.

Qualification does not prove that a model is scientifically best.

Primary-backend selection must use predeclared operational eligibility, not whichever backend appears to win an early architecture comparison.

22. Power Analysis and the Statistical Unit

Power analysis asks:

How many useful cases are needed to detect a meaningful difference with reasonable reliability?

The calculation considers:

- the number of scorable semantic-opportunity datasets;
- expected effect size;
- variation between datasets;
- acceptable error rates; and
- multiple planned comparisons.

The dataset is the primary statistical unit.

Why not treat every field as fully independent?

One dataset may contain 500 related fields while another contains five. Counting every field as an independent sample could make the large dataset dominate the result and exaggerate the amount of evidence.

The main comparison therefore gives datasets equal weight. Field-level details can still be reported as supporting information.

23. The NDP-50 External-Validation Track

NDP-50 is not the whole project. It is a separate external-validation track built on the same deterministic product and research controls.

Its purpose is to ask:

How does the system behave on a bounded sample from a real, heterogeneous external catalog?

NDP is a metadata catalog, not one uniform data repository.

Two units must remain separate:

Unit	Meaning
Dataset record	One catalog entry
Resource	A file, service, archive, or link attached to the record

Fifty dataset records do not necessarily mean fifty files.

24. Building the NDP Candidate Frame

The project first froze a complete lightweight snapshot of 5,823 NDP catalog records.

The lightweight frame contains enough information for fair selection, such as:

- stable dataset identity;
- title;
- organization;
- modification time; and
- indexed resource-format values.

It deliberately avoids expanding every resource detail before selection.

Why?

If detailed information were collected first and only easy datasets were retained, the study could silently favor accessible, supported, or clean records.

Every eligible identity remains in the frame. Later access failures, unsupported formats, missing resources, and catalog problems remain valid study outcomes.

25. Selecting and Splitting the NDP-50

The 50 selected dataset records are divided into:

Split	Count	Purpose
Development	15	Discover general capability problems and build general fixes
Validation	10	Check whether development fixes generalize
Sealed test	25	Produce the final held-out result after all gates pass

Selection uses frozen deterministic rules, including:

- format strata;
- organization caps;

- registered seeds;
- SHA-256-derived priority;
- separate split-assignment priority; and
- frozen reserve order.

The purpose is to avoid hand-picking easy datasets or allowing one dominant catalog organization to represent the whole sample.

Random selection within this bounded snapshot does not prove that the 50 records represent all NDP data or all scientific repositories.

26. What Each NDP Split May Be Used For

26.1 Development: 15 Datasets

Development cases may reveal general product problems.

They may support:

- a general parser fix;
- a new bounded rule;
- a security limit;
- a minimized regression fixture; or
- a documented capability change.

They may not support a special rule that recognizes one dataset by name.

26.2 Validation: 10 Datasets

Validation cases may:

- test whether a development fix is general;
- reject a proposed fix;
- test a frozen threshold; or
- reveal that the product is not ready.

They may not be used for dataset-specific tuning.

26.3 Sealed Test: 25 Datasets

Test identities and details remain sealed until:

- governance is approved;
- product and evaluation code are frozen;
- prompts and backend registry are frozen;
- gold workflow is complete;
- power and analysis plans are frozen;
- preregistration is verified; and
- independent test-release authorization passes.

The sealed test is the final exam, not another development round.

27. Capability Debt

Capability debt means a limitation in what the current product can reliably handle.

The NDP development loop uses a fixed taxonomy, including:

- access or transport;
- archives or multi-resource organization;
- unsupported format;
- conflicting format signals;
- parser or standard limitations;
- bounded-sample instability;
- declared-versus-observed conflicts;
- unit, time, space, or relationship limitations;
- evidence or provenance problems;
- performance, resource, or security limits; and
- documentation or prompt-injection risks.

An accepted fix requires:

- a general rule;
- a minimized regression fixture;
- validation evidence; and
- a change record.

Test cases may not be used to tune the product.

28. Declared Versus Observed

The NDP catalog may declare:

- format;
- MIME type;
- file size;
- field dictionary;
- temporal coverage;
- spatial coverage;
- license;
- access rights; and
- documentation.

The actual resource may agree or disagree.

The project keeps two values separate:

- declared: what the catalog says;
- observed: what direct inspection finds.

Possible comparison outcomes include:

- agreement;
- conflict;

- missing declaration;
- untestable; and
- not applicable.

NDP metadata is evidence, not automatic gold.

29. The CPA Sub-Study

CPA here refers to column property annotation.

For an applicable table, the task asks what property a column represents.

Examples:

Column	Possible property
city_name	City
temp_c	Air temperature
station_lat	Latitude
patient_id	Patient identifier

The sub-study also examines stability across:

- prompt wording;
- example-selection methods;
- scientific domains;
- vocabularies; and
- eligible model backends.

CPA is performed only when the tabular resource and target relationship are applicable. It is not forced onto every NDP dataset.

30. Why the Project Has Human Gates

Software can check hashes and file structure, but some decisions require qualified humans.

Examples include:

- whether a license allows the intended use;
- whether model transfer is permitted;
- which semantic vocabulary is appropriate;
- whether an annotation is scientifically correct;
- whether Swathi's feedback was fully addressed;
- whether a test release should be authorized; and
- whether a public preregistration matches the local frozen package.

A gate means:

The next stage remains locked until the required human evidence exists and passes validation.

A generated template is never counted as a completed human decision.

31. The Major Human and External Gates

Gate	Purpose
Data-governance review	Decide license, attribution, transfer, access, and redistribution policy
Vocabulary discovery and decision	Build and freeze the allowed semantic labels
Swathi feedback sign-off	Check whether F01-F16 were rigorously addressed
Source approval	Approve which evidence sources annotators may use
Annotator calibration	Confirm the exact annotator pair can apply the handbook
CPA applicability screening	Decide which cases are valid for the CPA task
Independent semantic gold	Produce the blinded reference standard
Execution freeze	Lock prompts, serialization, demonstrations, code, backends, and run rules
Power freeze	Lock the power policy and confirm pretest assurance
External preregistration	Publish an immutable pre-result protocol record
Test-release authorization	Witness and approve opening the sealed test
Test execution	Run the frozen test and preserve every required artifact

Several gates require people who are independent, non-developers, or different from earlier reviewers. One person cannot automatically fill every role.

32. Why Gate Order Matters

The correct order is:

```
write the rules
-> obtain review
-> freeze files and hashes
-> preregister
-> authorize test opening
-> execute once
-> analyze using the frozen plan
-> report all registered outcomes
```

The dangerous order would be:

```
look at the final result
-> choose the most favorable metric
-> change exclusions
-> call the new plan "original"
```

The gate architecture prevents that behavior.

Content changes invalidate downstream hashes that bind the changed content. A reviewer's approval of one version does not automatically approve a later version.

33. Freeze, Hashes, and Content Addressing

A freeze locks an exact version of a research object.

The project uses SHA-256 hashes as digital fingerprints for:

- source files;
- task bundles;

- prompts;
- response schemas;
- backend records;
- reviewer packets;
- human submissions;
- manifests;
- analysis plans; and
- validation receipts.

If a file changes, its fingerprint changes.

Hashes do not prove that a scientific decision is correct. They prove which exact content was reviewed, executed, or returned.

This distinction matters:

Question	Answered by
Did the file change?	Hash
Is the file scientifically correct?	Qualified review and evidence
Does the file follow the required structure?	Validator
Was the decision made before results were visible?	Timestamps, workflow, and receipts

34. Preregistration

Preregistration means recording the research plan before inspecting final results.

It includes:

- research questions;
- sampling rules;
- exclusions;
- primary metrics;
- statistical methods;
- backend conditions;
- missing-data policy;
- sensitivity analyses;
- stop rules; and
- reporting requirements.

A local file is not, by itself, external preregistration.

The project requires:

- an immutable public record such as OSF or Zenodo;
- an exported receipt;
- content hashes;
- timestamps; and
- independent verification that the public record matches the intended package.

The purpose is to show that the main rules were not invented after seeing the answer.

35. Test-Release Authorization

Even after readiness checks pass, the sealed test does not open automatically.

The planned release requires:

- a passing `test_ready=true` state;
- a study-operator signature;
- a distinct independent non-developer monitor;
- a completed release authorization; and
- validator-replayed receipts.

Developers and project collaborators cannot pretend to be the independent monitor.

The purpose is to create a witnessed boundary between preparation and final testing.

36. The Frozen Artifact Paper and Project History

The repository contains an earlier frozen Artifact Paper track.

Phases 1-11 built:

- the initial code and data model;
- a small internal pilot corpus;
- hand-built gold schemas;
- deterministic baselines;
- early evidence-constrained semantic experiments;
- a controlled retrieval experiment;
- benchmark tables and figures;
- a paper draft; and
- a GUI demonstration.

Those historical benchmark artifacts are frozen.

Later engineering work may improve the product, but it may not silently rewrite the old benchmark and keep calling it the original result.

37. Post-Freeze Engineering: Phases 12-20

Phases 12-20 expanded the deterministic substrate.

They added:

- a central extraction API;
- a capability registry;
- structured failure and abstention;
- a temporal subsystem;
- NetCDF/CF support;
- Zarr support;
- Parquet/Arrow support;
- JSON support;

- XML/XSD support;
- a unified schema envelope;
- categorized evaluation;
- read-only agent exports;
- GUI scratch extraction;
- continuous integration;
- pinned dependencies; and
- release-readiness checks.

These phases are engineering evidence. They are not automatically new headline research evidence.

Controlled challenge-pack success shows that declared controlled cases work. It does not prove broad ecological robustness across all scientific data.

38. Agent-Ready, Not an Autonomous Data Agent

The output is designed so that a future AI agent can:

- inspect the schema;
- inspect evidence;
- inspect conflicts;
- request a rerun;
- compare claims; and
- propose noncanonical suggestions.

The agent may not:

- replace the registered parser;
- mutate canonical claims;
- promote weak evidence into truth;
- create physical fields;
- overwrite explicit metadata; or
- replace required human decisions.

The purpose is to make the substrate useful to agents without allowing an agent to become the unaccountable source of truth.

39. Why the Product and the Study Stay Separate

The product asks:

Can we reliably extract an evidence-grounded schema from supported resources?

The study asks:

What do limited semantic reasoning and deterministic verification add?

The product may succeed even if the semantic study finds no benefit.

The study accepts several valid outcomes:

1. B adds useful semantic coverage at bounded risk;
2. C reduces risk at an acceptable coverage cost;

- 3. D has enough advantage to justify its complexity;
- 4. deterministic-only A is sufficient;
- 5. current semantic backends add no reliable value;
- 6. the comparison is underpowered or non-comparable.

A clean null result is still a real scientific result.

40. Repository Map

The code repository is organized like a laboratory.

Location	Purpose
extractors/	Format-specific deterministic readers
data/	Machine-readable experiment artifacts, manifests, reports, and workflows
docs/	Protocols, explanations, risk registers, review guides, and status reports
tests/	Automated regression and invariant checks
testdata/	Small software-test fixtures, not the final blind corpus
paper/	Manuscript source, references, and result macros
templates/	Neutral forms and receipt templates
gui_demo/	Demonstration interface
ci/	Automated cross-platform checks

Important top-level modules cover:

- extraction orchestration;
- unified schemas;
- time and unit handling;
- semantic architectures;
- architecture evaluation;
- blind sampling;
- gold workflows;
- power analysis;
- NDP acquisition and execution;
- assignment distribution;
- readiness gates;
- test execution; and
- statistical replay.

Software tests prove that implementation contracts behave consistently. They do not prove the scientific hypothesis.

41. The Project’s Current State

41.1 Built or Structurally Ready

- The deterministic extraction framework exists.
- Multiple registered format extractors exist.

- Evidence, provenance, conflict, partial, failure, and abstention structures exist.
- The unified schema envelope exists.
- The exact A/B/C/D architecture contracts exist.
- B/C response replay is enforced.
- Blind sampling, gold, calibration, power, inference, and preflight infrastructure exists.
- The NDP catalog snapshot contains 5,823 records.
- Fifty NDP datasets have been selected as 15 development, 10 validation, and 25 sealed test records.
- NDP structural development and validation work has been completed.
- The 25-case test split remains sealed.
- Human assignment, least-access packet, and validator infrastructure exists.
- Swathi's exact feedback-review packet has been delivered.

41.2 Still Incomplete

- Swathi has not yet completed the F01-F16 sign-off.
- Final data-governance policy approval is incomplete.
- Independent vocabulary discovery and decision are incomplete.
- Final annotator calibration is incomplete.
- Independent semantic gold is incomplete.
- Final backend registry and primary backend freeze are incomplete.
- Final power-policy freeze is incomplete.
- External preregistration has not yet completed independent verification.
- Independent test-release authorization has not occurred.
- The 25 sealed test datasets have not been executed.
- No final NDP-50 test result exists.
- The overall study may not yet be described as complete.

Infrastructure readiness is not the same as completed human evidence.

42. Where Swathi's Review Fits

Swathi's role is:

```
Human governance
-> feedback-response review
-> protocol improvement
-> preregistration readiness
-> eventual test-release readiness
```

She is serving as a postdoctoral project collaborator, not as an independent external validator.

Her task is to:

- review F01-F16;
- check whether the Phase 1 feedback was correctly understood;
- inspect whether the promised changes really appear in the research materials;
- identify gaps, inconsistencies, or oversights;
- recommend further changes where necessary; and
- return a completed collaborator-review payload.

She is not being asked to:

- run the NDP-50 experiment;
- create the independent gold by herself;
- serve as the independent test monitor;
- unseal the test set;
- select the final model; or
- approve every human gate.

Her review is important, but it is one quality gate within the larger architecture.

43. A School-Exam Analogy

The whole project can be understood as designing and running a fair exam.

Project component	School-exam analogy
Scientific data file	A textbook written in an unfamiliar language
Format detection	Identifying the language and type of book
Capability registry	List of qualified translators
Extractor	Translator for one file format
Physical schema	Chapters, pages, and sentences that really exist
Logical schema	The role each section plays
Semantic schema	What each section means
Units and temporal layer	Measurement and time rules
Evidence	The lines that support a translation
Provenance	Who translated it, when, and with which method
Unified envelope	One standard report format
Architecture A	Rules-only student
Architecture B	Rules plus one AI whole-book interpretation
Architecture C	The same AI answer checked by strict rules
Architecture D	Many smaller AI questions and a complex merge
Gold	Answer key made by independent teachers
Blind study	Teachers do not see student answers, and students do not see the key
Development split	Practice exam
Validation split	Check that the teaching method generalizes
Sealed test split	Final exam
Human gates	Permissions required before the exam begins
Preregistration	Publishing the scoring rules before grading
SHA-256	Digital fingerprint of each exam document
Paper	Final report of the exam and its results

44. Compact Glossary

Term	Plain-English meaning	Purpose
Schema	Description of data structure and meaning	Make data understandable
High-fidelity	Faithful to source evidence	Reduce unsupported guesses
Deterministic	Same input and settings produce the same result	Reproducibility
Extractor	Format-specific file reader	Observe real structure
Registry	List of supported capabilities	Controlled routing
Claim	A statement about the data	Unit of reasoning and evaluation
Evidence	Support for a claim	Auditability
Provenance	History of how a claim was produced	Traceability
Unknown	Not enough evidence	Honest uncertainty
Conflict	Evidence disagrees	Preserve disagreement
Partial	Some work succeeded	Avoid hiding incomplete extraction
Abstention	System refuses to guess	Safety and trust
Gold	Human reference answer	Evaluate system output
Blind	Outcome-relevant information is hidden	Reduce bias
Calibration	Practice round for annotators or systems	Confirm readiness
Backend	Exact model-running configuration	Reproducible semantic execution
Power analysis	Estimate required sample size	Avoid weak conclusions
Freeze	Lock an exact content version	Prevent post-result changes
Manifest	Machine-readable inventory	Check completeness
SHA-256	Digital fingerprint	Detect content changes
Validator	Rule-based checker	Enforce contracts
Receipt	Record that validation or delivery occurred	Audit trail
Preregistration	Public pre-result research plan	Limit post-hoc decisions
Human gate	Required human evidence before the next stage	Governance
NDP-50	External validation sample of 50 NDP records	Test external behavior
CPA	Column property annotation	Study column-level semantics
Sign-off	Completed and validated review decision	Formal evidence

45. Final Summary

The project is not trying to make an AI say as much as possible.

It is trying to build a system that:

- reads real scientific files with format-aware deterministic software;
- keeps physical structure separate from interpretation;
- attaches evidence and provenance to important claims;

- shows uncertainty, conflict, partial results, and unsupported features;
- allows only bounded semantic reasoning;
- checks semantic output without creating new physical fields;
- evaluates architectures against independent blind human gold;
- uses dataset-level statistics and frozen analysis rules;
- tests external behavior through NDP-50;
- protects the sealed test split;
- requires qualified human decisions at the correct time; and
- reports null, negative, failed, underpowered, and non-analysable outcomes honestly.

The complete logic is:

```
real resource
-> deterministic structure
-> evidence-grounded claims
-> optional bounded semantic reasoning
-> deterministic verification
-> independent blind gold
-> frozen statistical comparison
-> NDP-50 external validation
-> human and external approval gates
-> complete reporting
```

The project's central rule is:

When the system does not know, it may say that it does not know. A conflict must remain visible. A model cannot create a physical field. Test information cannot be used during development. Passing software tests cannot replace scientific evidence.

Source Documents in This Repository

This walkthrough is a plain-language synthesis of the project's current documents, including:

- README.md;
- docs/project_abstract.md;
- docs/project_convergence_2026-07-15.md;
- docs/file_in_to_schema_out_process_details_en.md;
- docs/semantic_architecture_research_protocol_v1.md;
- docs/ndp50_protocol_v1.md;
- docs/ndp50_semantic_cpa_protocol_v1.md;
- docs/ndp50_human_external_gate_ledger_v1.md; and
- docs/ndp50_swathi_feedback_signoff_guide_v1.md.

This walkthrough is explanatory documentation. It does not replace the frozen protocols, machine-readable manifests, validator receipts, or completed human decisions.